

Git & Version Control

Study Summary – Chapters 1–9

FWDN 232: Web Developer Tools | Full Stack Web Developer

Textbook: *Jump Start Git*, 2nd Edition (2020) — Shaumik Daityari

Ch1: Introduction to Version Control • Ch2: Getting Started with Git

Ch3: Branching in Git • Ch5: Git Workflows

Ch6: Correcting Errors • Ch7: Unlocking Git's Full Potential

Ch8: Integrate Git in Your Development Cycle • Ch9: Git GUI Tools

(Chapter 4 not included in source material)

Includes 15 practice MCQs with answer key

Prepared: June 2026

Chapter 1: Introduction to Git & Version Control

Key Definitions

Version Control (VCS): A system that records changes to a file or group of files over time, so you can review or **revert** to specific versions later. Think of it as taking **snapshots** / **checkpoints** of your work.

Backup vs Version Control: Version control **removes** the need for duplicate-named backup files, but it does **NOT** replace a real backup solution (e.g. external drive or cloud) that protects against disk failure.

Centralized VCS: A single copy of the project is hosted on a central server that everyone connects to; uses a 'first come, first served' principle. Examples: CVS, Subversion (SVN).

Distributed VCS: Every developer has a full copy of the entire project, including its complete **history**. Developers work and **commit** offline, then **synchronize** changes later. Examples: Git, Mercurial, Bazaar.

Git: A **distributed** version control system created by Linus Torvalds (Junio Hamano is the primary maintainer). Designed to be **distributed**, reliable, fast, and safe from **corruption**.

Git vs GitHub: Git is the version control system itself. GitHub is a cloud-based hosting service that stores Git **repositories** and adds a web interface. They are **NOT** the same thing.

Conflict: Occurs when multiple people change the same file in clashing ways. The VCS highlights the **conflict** and lets you resolve it, so **no** data is **lost**.

Exam Tip: Git was built by Linus Torvalds (the Linux creator) because BitKeeper, though good, was a commercial product unsuitable for an open-source project like Linux.

Exam Tip: **Distributed** VCS lets you work offline and **sync** directly with peers, bypassing the central server entirely.

Exam Trap: **Do NOT** confuse VCS (version control system, the general category) with CVS (Concurrent Versions System, just one specific tool).

Exam Trap: Version control is **NOT** a backup solution. 'Backup' here means avoiding duplicate files — you still need real off-site backups against disk failure.

Chapter 2: Getting Started with Git

Key Definitions

Repository: The parent **directory** containing your project — all files and directories **tracked** by Git. Created by running `git init`, which adds a hidden `.git` **directory**.

The Three Basic Operations: Git performs three core operations on your project: **track** (tell Git to watch a file), **stage** (mark changes to record), and **commit** (save a **snapshot**).

Tracked vs Untracked: A **tracked** file is one Git monitors for changes. New files are **untracked** by default and are ignored until you explicitly run `git add` to **track** them.

Staging: Tagging certain (or all) changes that you want to record in the next **commit**. `git add` moves changes into the **staging** area.

Commit: A **snapshot** that records the current state of your code since the last **commit**. Each **commit** has a unique **commit hash**, the author details, a message, and the list of changes. You can **revert** to any **commit**.

Commit Hash: A unique identifying signature generated automatically for each **commit** (40 characters). Usually the first 5–6 characters are enough to identify it; 3 characters is too ambiguous.

git diff: Shows the changes in **tracked** files since the last **commit**. Lines added are shown with + (green), lines **removed** with - (red).

Remote: A copy of your **repository** stored elsewhere (a server, the cloud, or a peer's machine). **origin** is the conventional name for the default **remote**.

.gitignore: A file in the **repository** root that lists files, directories, or extensions Git should **never track** (e.g. `*.exe`, `.DS_Store`, `node_modules`). Patterns use wildcards like `*.exe`.

Essential Commands

```
git config --global user.name "Name" # set identity for all repos
```

```
git init # initialize a repository
```

```
git add . # stage all | git add file # stage one file
```

```
git commit -m "message" # create a commit
```

```
git status | git log | git show
```

```
git remote add origin && git push -u origin master
```

Exam Tip: The package-manager mantra: it installs an OLDER but MORE RELIABLE version of Git. Memorize that trade-off.

Exam Tip: 'Why git add again?' — like packing a parcel: `git add` = put item in the box, `git commit` = seal & label it, `git push` = send it. You re-add because you choose WHICH changed files to **commit**.

Exam Trap: `git add` does **NOT** create a **commit** — it **only** stages. Changes are **not** saved to **history** until `git commit` runs.

Exam Trap: The `-u` flag in '`git push -u origin master`' LINKS your **branch** to the **remote** for future reference; it does **not** 'upload urgently'.

Chapter 3: Branching in Git

Key Definitions

Branch: An independent path of development — effectively a new copy of the project you can experiment on without affecting the original (master) **branch**. A **branch** is a named sequence of **commits**.

master Branch: The default **branch** you start in. The name 'master' implies **no** superiority — it is just a convention for the main line of development.

HEAD: A pointer to the latest **commit** of the current active **branch** — i.e. the tip of the **branch**. **HEAD** and the tip of the active **branch** point to the same **commit**.

Cloning: The process of creating a local copy of a **repository** from a different source. After **cloning** you can also see **remote branches** with `git branch -a`.

Checkout: Switches the active **branch**. `git checkout` moves to an existing **branch**; `git checkout -b` creates and switches in one step.

Merge: Combines the changes of one **branch** into another. You **must** first **checkout** the destination **branch**, then run `git merge`.

Fast-Forward Merge: When the target **branch** has **no** new **commits** of its own, Git simply moves the **branch** pointer forward. **Only** the pathway changes and **HEAD** is updated — **no merge commit** is created (this is the default).

No-Fast-Forward Merge (--no-ff): **Forces** Git to create a new **merge commit** on the base **branch** even when **fast-forward** is possible. Generally recommended for **merges** into master to logically record the feature.

Merge Conflict: Happens when the same file is modified in non-common **commits** in both **branches**. Git raises a **conflict** so you **do not lose** data and asks you to resolve it manually.

Branch Commands

```
git branch          # list | git branch -a  # include remotes
git branch test     # create | git checkout test  # switch
git checkout -b feature  # create + switch in one command
git branch -m newname  # rename current branch
git branch -d name    # safe delete | git branch -D name  # force delete
git checkout master && git merge feature
```

Exam Tip: `-d` (safe) **only deletes** a **branch** already merged/synced; `-D` (capital) **FORCE-deletes** even unmerged **commits** with **no warning**.

Exam Tip: Use `--no-ff` when **merging** into master so the **history** clearly shows where a feature was integrated.

Exam Trap: A **fast-forward merge** does **NOT** create a **merge commit** — it just moves the pointer. **Only** `--no-ff` creates a dedicated **merge commit**.

Exam Trap: `git branch` creates a **branch** but does **NOT** switch to it — you remain on the current **branch** until you **checkout**.

Chapter 5: Git Workflows

Key Definitions

Git Workflow: A set of guidelines a team follows to manage a project — covering the architecture, how contributions are made, and how others' work is **merged** in.

Centralized Workflow: The simplest **workflow**: a single **branch** (usually master) is used for all operations, treated as the main copy. Also called the 'trunk' **workflow** (SVN's master is called trunk). **No branching** is used.

Feature-Branch Workflow: Each new feature is developed in its own separate **branch**, then **merged** or rebased into master after a code review. A developer **must never** directly **commit** to the local master **branch**.

Gitflow Workflow: An extension of the feature-branch **workflow** with defined **branch** roles. Two core **branches**: develop (latest development) and master (last stable release), plus feature, release, and hotfix **branches**.

Hotfix Branch: In Gitflow, a **branch** created from master to fix a **critical bug** immediately. Once fixed it is **merged** back into BOTH master and develop.

Forking Workflow: A cloud implementation of the feature-branch **workflow**. Each developer has a personal server-side copy (a **fork**); changes are **pushed** to the **fork** and **merged** via a **pull** request after review.

Fork: A developer's personal copy of the central **repository** in the cloud. **origin** points to the **fork**; **upstream** points to the central (main) **repository**.

Choosing a Workflow

Workflow	Best For	Branching?
Centralized	Beginners, solo / personal projects, SVN converts	No
Feature-branch	Growing teams, open source (enables code review)	Yes
Gitflow	Mature projects, mixed-skill teams, multiple released versions	Yes (roles)
Forking	Any cloud-hosted / open-source project	Yes (+ fork)

Exam Tip: Open-source projects **must** use at least a feature-branch **workflow** so code reviews happen before contributor **merges**.

Exam Tip: In Gitflow, developers treat the develop **branch** as a 'pseudo master' and are essentially **never** directly involved with the real master.

Exam Trap: The **centralized workflow** does **NOT** use **branching** — giving everyone master access means one bad **commit** can **corrupt** the whole **repository**.

Exam Trap: In Gitflow a hotfix **merges** into BOTH master AND develop — forgetting develop means the **bug** reappears in the next release.

Chapter 6: Correcting Errors While Working with Git

Key Definitions

Undo git add (untrack): `git rm --cached` untracks a file but leaves it on disk. Use it to undo accidentally **tracking** a new file before it is **committed**.

git reset --soft: Undoes a **commit** but **KEEPS** its changes **staged** for review. `HEAD~1` means go back one **commit** from the current **HEAD**.

git reset --mixed (default): Undoes the **commit** AND unstages the changes (files keep their edits but are **not staged**). This is the default **reset** mode.

git reset --hard: Undoes the **commit** and **discards** all changes — takes you to a state even before the files were edited. **Avoid** it unless you are absolutely sure; it **permanently** destroys work.

reset vs revert: **reset** **rewrites history** by moving **HEAD** backward. **revert** creates a NEW **commit** that reverses the changes of a **faulty commit** — **history** is preserved. **revert** is safe for shared/pushed **commits**.

git commit --amend: Changes the message (or content) of the LAST **commit**. **Note** the **commit hash** changes too, effectively **rewriting history**.

Undo a Push (force push): To **remove** a **pushed commit** you **reset** locally then **force** the update with `git push -f`. A normal **push** is **rejected** because the **remote HEAD** is ahead of your local **branch**.

git blame: Shows, for each line of a file, the **commit hash**, author, and date that last changed it. Used when you **KNOW** which file contains the **bug**.

git bisect: Performs a binary search through **commits** to find the one that introduced a **bug**. You mark a known 'good' **commit** and a 'bad' **commit**; Git narrows down in logarithmic steps. Used when you **DON'T** know the source.

Reset Modes — Quick Reference

Option	Commit	Changes Staged?	Changes Kept?
--soft	Undone	Yes (staged)	Yes
--mixed (default)	Undone	No (unstaged)	Yes
--hard	Undone	No	No (discarded)

```
git rm --cached file | git reset --soft HEAD~1 | git revert
```

```
git commit --amend -m "new message" | git push -f origin master
```

```
git blame file | git bisect start | git bisect reset
```

Exam Tip: `git bisect` can find a bad **commit** among thousands in just a few steps because it uses **BINARY SEARCH** (logarithmic). You can automate it: `git bisect run .`

Exam Tip: Use **revert (not reset)** to undo a **commit** that has already been **PUSHED** — it adds a new reversing **commit** instead of **rewriting** shared **history**.

Exam Trap: `--hard` **PERMANENTLY discards** your changes. The book explicitly warns to **avoid** it unless you are absolutely certain.

Exam Trap: **reset** **CHANGES history**; **revert** **PRESERVES** it. **Do not** use **reset** on **commits** others have already **pulled**.

Chapter 7: Unlocking Git's Full Potential

Key Definitions

git log --oneline: Compresses each **commit** to a single line (short **hash** + message), fitting many more **commits** on screen. Combine with **--all**, **--decorate**, and **--graph** to visualize all **branches**.

git shortlog: Groups **commits** by author, showing each contributor and their **commit** messages. Postfix **-n** to sort by number of **commits** instead of by name.

Filtering log: **git log -n 2** shows the last 2 **commits**; **--after/--before** (or **--since/--until**) filter by date; **--author='Name'** filters by author; **--grep** searches **commit** messages.

Tag: A label attached to a specific milestone **commit**, typically a version number. Two types: lightweight (name + **commit only**) and annotated (name + tagger info + message).

Annotated vs Lightweight Tag: Annotated **tags** store the tagger, date, and a message — preferred in organizations. Lightweight **tags** are just a name pointing to a **commit** — handy for personal projects.

Ref & reflog: A **ref** (reference) is a pointer to a **commit** (**HEAD**, **ORIG_HEAD**, **MERGE_HEAD** are **refs**). The **reflog** is a '**log of refs**' recording every action locally — it can recover **lost commits**. It is **NOT synced** to the server.

git fsck: File system check — finds **commits** that are **not** part of any **branch (lost commits)**, which **git log cannot** show. Recover one with **git merge**.

Rebase: **Rewrites** a branch's **history** by moving it onto a new base **commit**, producing a **LINEAR history** without the loops a **merge** creates. Preferred for keeping a clean central **history**.

Squash: Combining multiple **commits** into one, done via interactive **rebase** (**git rebase -i HEAD~2**). There is **no** stand-alone '**git squash**' command.

Stash: Saves uncommitted changes and **reverts** the **working directory** to the last **commit**, so you can switch context. Reapply later with **git stash apply** (optionally **stash@{n}**).

git add -p: Stages **only PART** of a file's changes by splitting them into 'hunks'. Options include **y (stage)**, **n** (skip), **s** (split), **e** (edit). Lets one file's edits go into separate **commits**.

Cherry-Pick: Picks a single **commit** from another **branch** and applies it to your current **branch** — when a full **merge** or **rebase** would be too much. **git cherry-pick**.

Power Commands

```
git log --oneline --graph --all --decorate
```

```
git log --author="Name" | git log --grep="test" | git log -n 2
```

```
git tag -a v1.0 -m "msg" | git push origin --tags
```

```
git rebase master | git rebase -i HEAD~2 | git pull --rebase origin master
```

```
git stash | git stash apply stash@{1} | git cherry-pick | git reflog
```

Exam Tip: **rebase** = **LINEAR history (no loops)**; **merge** = preserves the true **branch** topology with a **merge commit**. Prefer **rebase** to keep a central **repo** tidy.

Exam Tip: **reflog** and **fsck** are your recovery tools: a '**lost**' **commit** from a **hard reset** or **deleted branch** is usually still recoverable.

Exam Trap: There is **NO** '**git squash**' command — **squashing** is done through interactive **rebase** (**git rebase -i**).

Exam Trap: **Tags** are **NOT pushed** automatically with **git push** — you **must** run **git push origin --tags** (or **push** a specific **tag**).

Chapter 8: Integrate Git in Your Development Cycle

Key Definitions

DevOps: A cross between Development, Operations, and QA that automates the software development cycle to lower time to market. Arose to **remove** the friction between separate dev/test/deploy/maintenance teams.

Continuous Integration (CI): The second step of the DevOps cycle: testing all code on every **push** and **merging** into the main codebase if all tests pass. Ensures new changes **do not** break existing functionality.

Git Hook: A custom script that executes automatically when a predefined Git event occurs. Lets you trigger actions at key points in the development life cycle. Stored in the `.git/hooks` **directory**.

Client-side vs Server-side Hooks: Client-side **hooks** react to local actions (pre-commit, commit-msg, post-commit). Server-side **hooks** react to network actions like a **push** (pre-receive, update, post-receive).

pre-commit Hook: The most **important hook** — runs before a **commit** message is entered, commonly used to run unit tests so broken code is **never committed**.

Travis CI: A popular hosted CI tool for GitHub **repositories** (free for open source). On every **push** it **clones** the **repo** to a cloud VM, installs requirements, and runs predefined unit tests.

.travis.yml: The configuration file (in the **repo** root) that tells Travis CI the language, versions, install commands, and test commands. Each version listed creates a separate build.

Common Git Hooks

Hook	Triggered	Side
pre-commit	Before the commit message is entered	Client
commit-msg	With the commit message	Client
post-commit	After a commit completes	Client
pre-push	Before changes are sent to a remote	Client
pre-receive / post-receive	Before / after a push from a client	Server

Exam Tip: **Hooks** live in `.git/hooks` as `.sample` files. To activate one, **remove** the `.sample` extension and make sure the file is executable.

Exam Tip: Travis CI **only** tests **repositories** hosted on GitHub; for other hosts you work around it with submodules.

Exam Trap: Git **hook** sample files are Bash scripts (`#!/bin/sh`), but you **CAN** change the first line to use another language — they are **not** limited to Bash.

Exam Trap: Continuous integration is the **SECOND** step of DevOps, **not** the first. Continuous development comes first.

Chapter 9: Git GUI Tools

Key Definitions

GUI Tool: A graphical client used instead of the terminal. Easier for beginners, but tools differ in UI and terminology, lack some terminal power, and run more slowly than commands.

GitHub Desktop: GitHub's own unified GUI client (Windows + macOS, since 2015). Simplifies the **workflow** by hiding the term '**staging**' — you just select files and **commit**. Focused on bridging local repos with the GitHub website.

Sourcetree: Atlassian's GUI client (Windows + macOS). Works with both Git and Mercurial, offers more features and control, and its options closely match the terminal commands.

Submodule: A Git **repository** nested within a parent **repository**. Listed in the left menu of Sourcetree alongside **branches**, **tags**, **remotes**, and **stashes**.

Editor Integrations: Git can also be used through text editors: Atom (built-in Git/GitHub), Sublime Text (Git package), and Visual Studio's version control tools.

GUI Tool Comparison

	GitHub Desktop	Sourcetree
Developer	GitHub	Atlassian
Platforms	Windows, macOS	Windows, macOS
VCS support	Git	Git + Mercurial
Audience	Beginners (simplified terms)	Power users (matches terminal)

Exam Tip: Neither GitHub Desktop nor Sourcetree supports Linux. Cross-platform GUIs include SmartGit, **Fork**, and GitKraken (Windows/macOS/Linux).

Exam Tip: On a **remote** server (often a headless VM) **only** command-line Git works — which is why knowing terminal commands matters even if you use a GUI.

Exam Trap: GitHub Desktop deliberately hides the word '**staging**' to simplify things — **do not** assume the underlying **staging** step disappeared.

Exam Trap: GUI tools are **NOT** cross-platform and run slower than the terminal; **no** single GUI behaves identically across Windows, macOS, and Linux.

Practice Questions — Top 15 MCQ

Choose the single best answer. Options are deliberately close — read carefully. Answer key follows on the next page.

Q1. A developer needs to keep working offline for a week, committing changes without any server connection, then sync later. Which property of Git makes this possible?

- A. It is a centralized version control system
- B. It is a distributed VCS where each clone holds the full history
- C. It uses a first-come, first-served locking model
- D. It stores all commits only on the origin remote

Q2. You created a new file but have NOT committed it, and you realize Git should not be tracking it yet. Which command removes it from tracking while keeping the file on disk?

- A. `git rm mistake_file`
- B. `git reset --hard`
- C. `git rm --cached mistake_file`
- D. `git checkout -- mistake_file`

Q3. Which statement about a fast-forward merge is correct?

- A. It always creates a new merge commit
- B. It only moves the branch pointer forward and updates HEAD, with no merge commit
- C. It is triggered by the `--no-ff` flag
- D. It is the only way to merge when both branches have new commits

Q4. A commit has already been pushed to a shared remote and others have pulled it. You must undo its effect safely. What is the recommended approach?

- A. `git reset --hard` then `git push -f`
- B. `git commit --amend`
- C. `git revert` to add a new reversing commit
- D. `git rm --cached` on the affected files

Q5. Which reset mode undoes a commit but leaves the changes STAGED, ready to be re-committed?

- A. `git reset --hard HEAD~1`
- B. `git reset --mixed HEAD~1`
- C. `git reset --soft HEAD~1`
- D. `git revert HEAD~1`

Q6. You have no idea which of the last 500 commits introduced a bug, but you know a commit from last month was fine. Which tool finds the culprit most efficiently?

- A. `git blame`, because it shows line authorship
- B. `git bisect`, because it does a binary search through commits
- C. `git log --oneline`, scanning each commit by hand
- D. `git reflog`, because it lists every local action

Q7. In the Gitflow workflow, where is a hotfix branch created from, and where must it be merged back?

- A. Created from `develop`; merged into `master` only
- B. Created from `master`; merged into BOTH `master` and `develop`
- C. Created from a feature branch; merged into `develop` only
- D. Created from a release branch; merged into `master` only

Q8. What is the practical difference between an annotated tag and a lightweight tag?

- A. Lightweight tags store the tagger name, date, and message
- B. Annotated tags can only be created on the `master` branch
- C. Annotated tags store tagger info and a message; lightweight tags are just a name pointing to a commit
- D. There is no difference; the names are interchangeable

Q9. You ran git push but it was REJECTED because the remote HEAD is ahead of your local branch after a reset. What must you do to overwrite the remote (assuming you are sure)?

- A. Run git pull --rebase first, always
- B. Force the push with git push -f
- C. Run git commit --amend and push again
- D. Delete the remote and re-add it

Q10. Which statement about git stash is TRUE?

- A. Stashed changes are automatically pushed to the remote
- B. A stash saves uncommitted changes and reverts the working directory to the last commit
- C. git stash permanently deletes your uncommitted changes
- D. Stashes are stored on origin and shared with the team

Q11. A teammate wants only ONE specific commit from another branch — not the whole branch. Which command applies just that commit to the current branch?

- A. git merge
- B. git rebase
- C. git cherry-pick
- D. git checkout

Q12. Which choice correctly distinguishes Git from GitHub?

- A. Git is a website; GitHub is a command-line tool
- B. Git is a distributed VCS; GitHub is a cloud service that hosts Git repositories
- C. They are two names for the same product
- D. Git only works online; GitHub works offline

Q13. You want to combine your last three messy commits into a single clean commit before opening a pull request. What do you use?

- A. The stand-alone git squash command
- B. git merge --squash on master
- C. Interactive rebase: git rebase -i HEAD~3
- D. git reset --soft only

Q14. Which Git hook is most commonly used to run unit tests so that broken code is never recorded, and on which side does it run?

- A. post-receive, server-side
- B. pre-commit, client-side
- C. pre-receive, server-side
- D. commit-msg, client-side

Q15. Why does the book recommend learning terminal Git even if you prefer a GUI like GitHub Desktop or Sourcetree?

- A. GUI tools support every operating system identically
- B. GUI tools are always faster than the terminal
- C. On a headless remote server only command-line Git works, and GUIs aren't cross-platform
- D. Terminal commands cannot cause data loss, unlike GUIs

Answer Key

Correct answers with a one-line justification.

#	Answer	Justification
Q1	B	Every clone in a distributed VCS holds the full history, so you can commit offline and sync later.
Q2	C	git rm --cached untracks the file but leaves it in the file system; plain git rm would also delete it.
Q3	B	Fast-forward simply advances the pointer/HEAD with no merge commit; --no-ff is what forces a merge commit.
Q4	C	revert adds a new reversing commit and preserves shared history; reset --hard/-f rewrites history others already have.
Q5	C	--soft undoes the commit but keeps changes staged; --mixed unstages them; --hard discards them.
Q6	B	git bisect performs a binary search (logarithmic steps); blame is for when you already know the file.
Q7	B	A hotfix is cut from master and merged back into BOTH master and develop so the fix isn't lost next release.
Q8	C	Annotated tags carry tagger info + message (preferred in orgs); lightweight tags are just a name on a commit.
Q9	B	After a local reset the remote is ahead, so a normal push is rejected; -f forces the overwrite.
Q10	B	stash saves uncommitted work and restores the last-commit state; it is local and reversible, not pushed.
Q11	C	cherry-pick applies a single chosen commit; merge/rebase would bring the whole branch.
Q12	B	Git is the VCS itself; GitHub is a cloud hosting service built around Git.
Q13	C	Squashing is done via interactive rebase (git rebase -i); there is no stand-alone git squash command.
Q14	B	pre-commit runs client-side before the message is entered — the usual place to run unit tests.
Q15	C	Headless remote servers only have the CLI, and GUI tools are not cross-platform / behave differently per OS.